

TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB (IFCS03)

MÓDULO: PROYECTO. CURSO: 2024-2025

CONVOCATORIA: ORDINARIA

Título del proyecto: Gestión de Recordatorios

PROFESOR TUTOR: Nombre y Apellidos

Alumno:

Parra Llansó , Manuel	DNI: 06592686E
------------------------------	-----------------------

MODALIDAD DE PROYECTO:

Modelo b): Proyectos de gestión

ÍNDICE

1. JUSTIFICACIÓN.....	3
2. INTRODUCCIÓN.....	3
3. TOMA DE REQUISITOS.....	3
4. OBJETIVOS.....	4
5. DESARROLLO.....	5
5.1. METODOLOGÍA.....	5
5.2. TECNOLOGÍAS.....	5
5.3. PROTOTIPADO Y USABILIDAD.....	6
5.4. MODELADO DE DATOS.....	9
5.5. SEGURIDAD Y AUTENTICACIÓN.....	10
5.6. DISEÑO E IMPLEMENTACIÓN.....	10
6. DESPLIEGUE DEL PRODUCTO.....	13
7. PRESUPUESTO, RIESGOS, VIABILIDAD.....	13
8. DIFICULTADES ENCONTRADAS Y RESOLUCIÓN.....	14
9. CONCLUSIONES.....	15
10. REFERENCIAS BIBLIOGRÁFICAS.....	16

1. JUSTIFICACIÓN

Hoy en día mucha gente tiene dificultades para recordar muchas tareas a la vez lo cual hace útil esta aplicación ya que permite la creación de recordatorios como muchas otras aplicaciones pero lo que le diferencian es que además de poder crearlas puedes compartirla con otros usuarios para que ellos también sepan de la tarea como puede ser una reunión o la entrega de una tarea.

2. INTRODUCCIÓN

Este proyecto se trata de una aplicación que permite la creación de recordatorios que te avisarán dos horas antes de la fecha de que está asignado el recordatorio pudiendo asignarle una categoría específica al recordatorio previamente creada por el usuario. Otra característica que tiene esta aplicación es poder compartir recordatorios a los diferentes contactos que tengas agregados a tu lista de contactos para que les avise de cuándo sean dos horas antes del evento importante.

3. TOMA DE REQUISITOS

Este proyecto tendrá los siguientes requisitos:

Requisitos Funcionales

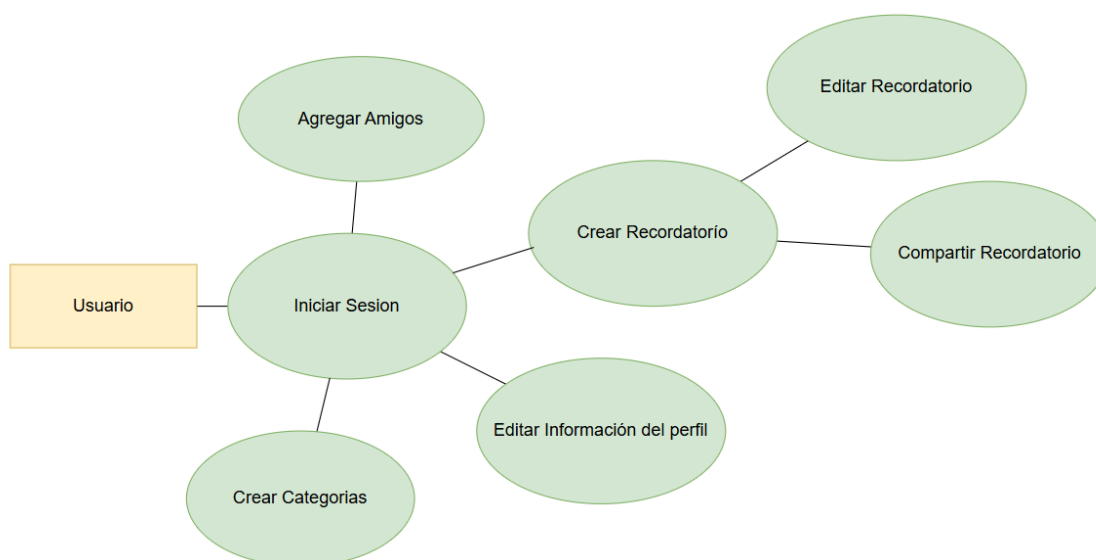
1. **Creación de recordatorios:** La aplicación debe permitir a los usuarios crear recordatorios personalizados con opciones de colores, fuentes e íconos.
2. **Edición de recordatorios:** Los recordatorios podrán editarse en el caso de que el creador del recordatorio habilite que se pueda editar.
3. **Compartición de recordatorios:** Los usuarios podrán compartir recordatorios con contactos agregados en la app.
4. **Organización avanzada:** Los recordatorios deberán permitir niveles de prioridad (alta, media, baja).
5. **Notificaciones:** La aplicación enviará notificaciones dos horas antes al creador y a los destinatarios.

6. **Sincronización de datos:** Los recordatorios deben sincronizarse automáticamente en todos los dispositivos del usuario mediante autenticación con cuenta.

Requisitos No Funcionales

1. **Rendimiento:** La aplicación debe ser rápida y eficiente
2. **Seguridad:** Los datos de los usuarios y recordatorios deben almacenarse de manera segura en la nube con autenticación .
3. **Interfaz intuitiva:** La UI/UX debe ser amigable y accesible para cualquier usuario, con un diseño limpio y simple.
4. **Compatibilidad:** La aplicación debe ser compatible con dispositivos Android a partir de la versión de Android 8.0 .
5. **Escalabilidad:** La base de datos y el sistema deben soportar el crecimiento del número de usuarios sin afectar el rendimiento.
6. **Disponibilidad:** La aplicación debe garantizar un tiempo de actividad del 99% cuando se use con conexión a internet.
7. **Consumo de batería:** La aplicación debe optimizar el uso de energía para no agotar la batería rápidamente.

A continuación muestro un esquema de los casos de usos:



Casos de uso

4. OBJETIVOS

Desarrollar una aplicación que permita a los usuarios crear recordatorios de tareas y compartir las en 3 meses.

Los plazos del proyecto serán los siguientes:

17/03 - 23/03: Análisis Inicial y planteamiento de la aplicación

24/04 - 6/04: Diseño de la aplicación

7/04 - 27/04: Desarrollo de la interfaz del proyecto

28/04 - 04/05: Desarrollo de la funcionalidad del login

05/05 - 18/05: Desarrollo de las funcionalidades de crear recordatorios y crear categorías

19/05 - 25/05: Desarrollo de las funciones sociales de la aplicación(Compartir recordatorios y Agregar amigos)

25/05 - 5/06: Ultimos ajustes de la aplicación

5. DESARROLLO

5.1. METODOLOGÍA

Voy a usar la metodología cascada ya que para el tipo de proyecto que voy hacer de gestión es mejor porque los requisitos no van ha cambiar prácticamente nada y esta metodología es bastante eficiente ya que va paso a paso y solo hay una entrega al final del proyecto.

5.2. TECNOLOGÍAS

Para el proyecto se usarán las siguientes tecnologías:

Interfaz y funcionalidades: He decidido para este apartado utilizar Kotlin porque es el lenguaje que actualmente es el recomendado por Google y cada vez se usa más para programar en dispositivos Android.

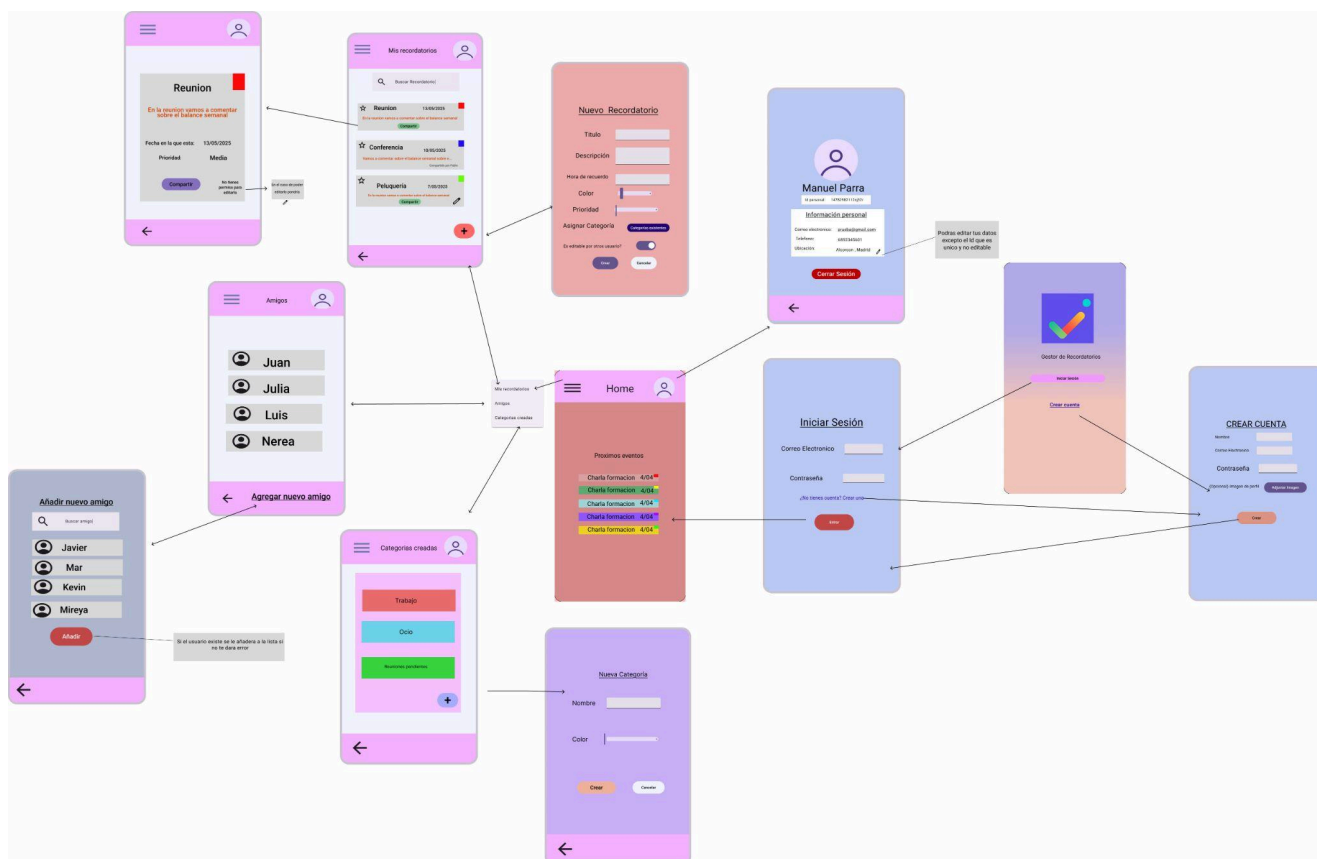
Base de datos: Voy a usar Firestore ya que es bastante óptimo para una base de datos en vivo para gestionar los recordatorios

Servidor en la nube: He elegido Storage para almacenar las imágenes de perfil ya que es un servidor fácil de usar y eficiente del mismo paquete de Firebase.

Autenticación de usuarios: Voy a usar Firebase Auth ya que pertenece al mismo kit de herramientas de los demás apartados y mantiene las contraseñas seguras ante otros usuarios

5.3. PROTOTIPADO Y USABILIDAD

El prototipo del diseño de la aplicación tiene la siguiente interfaz:



Prototipado del diseño de la aplicación

He decidido utilizar esta gama de colores pastel para que no sea ni muy vistosa ni muy sosa para el usuario. A continuación voy a ir pantalla por pantalla justificando las decisiones tomadas para acabar realizando ese diseño de pantalla:

Pantalla inicial: He elegido este diseño de pantalla ya que vi que a los usuarios se iban a sentir familiarizados con este diseño ya que en muchas aplicaciones como puede ser Spotify tiene este diseño de pantalla inicial sencilla y directa.

Pantalla de registro y pantalla de inicio de sesión: En estas dos pantallas he elegido este diseño simple de formulario para no abrumar al usuario con muchos detalles innecesarios y campos totalmente prescindibles que pueden ser rellenados posteriormente.

Pantalla de entrada: En esta pantalla he decidido este diseño siguiendo la misma estética de hacer un diseño simple con botones accesibles y fáciles de manejar. He decidido mostrar los recordatorios próximos en una columna con tarjetas porque me parece una forma fácil y rápida de tener accesible los recordatorios más cercanos.

Pantalla de perfil: He optado por este diseño ya que me parece sencillo de entender para el usuario que quiera revisar su información, editar su información personal y cerrar sesión.

Pantallas de recordatorios: En la pantalla donde muestro los recordatorios del usuario he decidido añadir un poco más de complejidad para que se puedan ver en un vistazo de la información importante del recordatorio y si necesitas más información pulsas en él y ya te lleva a una pantalla con más información acerca del recordatorio. Respecto a la hora de crear un nuevo recordatorio he decidido hacer un formulario con opción de rellenar toda la información y la opción de poder cancelar la creación del recordatorio.

Pantallas de categorías: En estas pantallas he decidido hacer un diseño simple para que sea fácil ver todas tus creadas y poder crear una nueva categoría rápidamente

Pantallas sociales: En estas pantallas he optado por diseños intuitivos para el usuario donde te muestran todos tus contactos y a la hora de añadir un nuevo amigo es sencillo de realizar ya que poniendo el nombre del usuario ya podrás añadir .

5.4. MODELADO DE DATOS

Para el proyecto he decidido que mi base de datos No-SQL tenga tres colecciones : **Users** ,**Categories**,**Notification** y dentro de la colección Users hay una subcolección en cada documento que guarda los recordatorios creados por el usuario llamada **Reminders**.

A continuación voy a mostrar el diagrama E/R con las tablas de mi base de datos y seguido una explicación de lo que guarda cada tabla junto una explicación de por cual campo se relacionan las tablas:

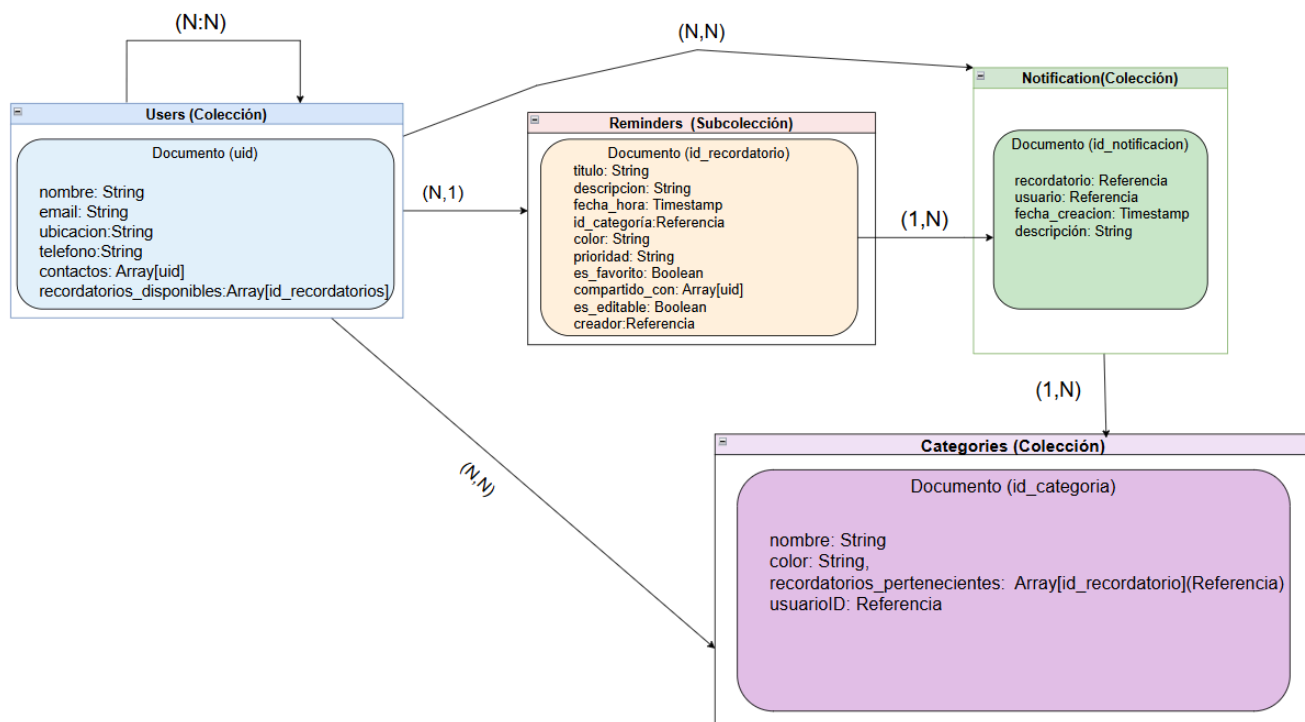


Diagrama E/R de tablas

Users: En esta colección se almacenará toda la información del usuario además de el array donde se almacenan los uid de los amigos agregados en la app y la subcolección de **Reminders**.

Reminders: En esta subcolección se almacenará todos los recordatorios creados por el usuario dentro de cada documento hay un array con los uid de las personas que se le ha compartido el recordatorio junto con un campo que muestra quien es el creador del recordatorio y la categoría que tiene si tiene alguna.

Categories: En esta colección se almacenará cada categoría con los recordatorios pertenecientes y el id del usuario que creó la categoría.

Notification: En esta colección está la información de cada notificación que se envía de aviso al recordatorio donde está la descripción de la notificación, el usuario a quien se le va a enviar y la fecha de creación de la notificación junto con el recordatorio del que está notificando.

5.5. SEGURIDAD Y AUTENTICACIÓN

En el proyecto he utilizado Firebase Authenticator para realizar una segura autenticación. He decidido que sea esta herramienta porque guarda en la nube las contraseñas y es fácil de manejar además es totalmente compatible con las demás herramientas utilizadas ya que pertenece al mismo conjunto de herramientas de Firebase.

5.6. DISEÑO E IMPLEMENTACIÓN

A continuación se presenta el diagrama de clases correspondiente a las entidades principales que estructuran el modelo de datos de la aplicación. Este modelo sigue un enfoque orientado a objetos y se apoya en clases de datos (data class) de Kotlin, en consonancia con la arquitectura MVVM empleada en el proyecto.

Las clases representadas son:

Recordatorio: Clase principal que encapsula los datos de un recordatorio personal o compartido.

Usuario: Representa a los usuarios registrados en la aplicación.

UsuarioAmigo: Representa los contactos del usuario, incluyendo referencias a otros usuarios y sus datos básicos.

El diagrama ilustra las relaciones lógicas entre estas entidades, las cuales se establecen mediante identificadores (String) o referencias (DocumentReference, en el caso de Firebase).

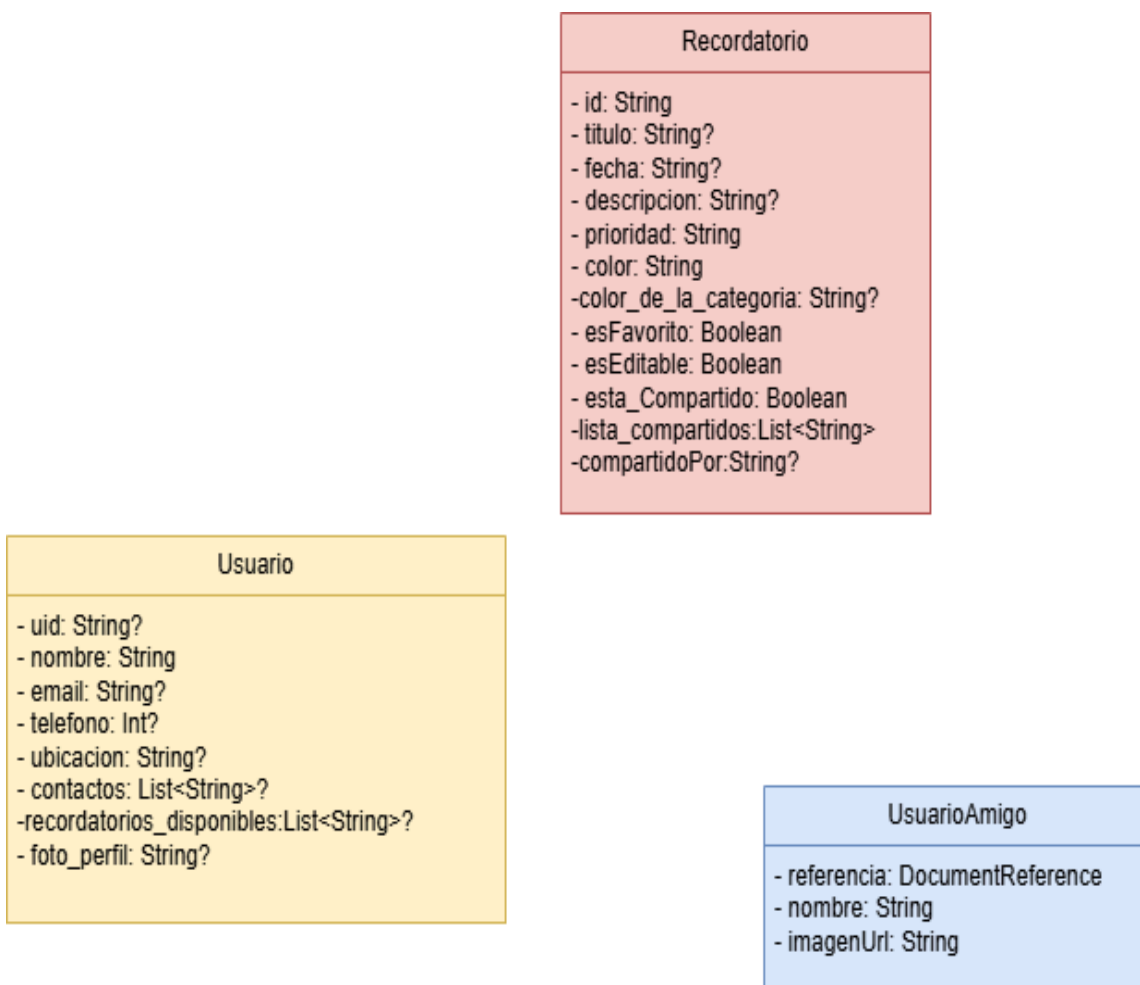


Diagrama de clases

Una parte a destacar en la código sería el login el cual tiene entre todos los métodos destacan el **onLoginClick**. Este método comprueba si está en blanco las credenciales al pulsar al botón y si esta en blanco te sale el texto que se muestra en la imagen si hay texto se intenta llevar a cabo el login y da error si la contraseña no es válida o si no existe el usuario además de mostrar el texto correspondiente al error.

```
fun onLoginClick() {  
    if (email.isBlank() || password.isBlank()) {  
        errorMessage = "Por favor, introduce un correo y una contraseña."  
        showErrorDialog = true  
        return  
    }  
  
    auth.signInWithEmailAndPassword(email, password)  
        .addOnCompleteListener { task ->  
            if (task.isSuccessful) {  
                loginSuccess = true  
            } else {  
                val firebaseError = task.exception?.message ?: "Error desconocido"  
                errorMessage = when {  
                    firebaseError.contains( other: "password is invalid", ignoreCase = true) -> "La contraseña no es válida."  
                    firebaseError.contains( other: "no user record", ignoreCase = true) -> "No existe una cuenta con este correo."  
                    else -> "Error al iniciar sesión. Intenta de nuevo."  
                }  
                showErrorDialog = true  
            }  
        }  
    }  
}
```

Método que comprueba si son correctas las credenciales e inicia sesión

Otra de las partes que destacaría es el método obtenerFotoPerfil() que usó para traer la foto de perfil a cada el cual consigue el uid del usuario y busca si subió la foto de perfil dependiendo de si está o no devuelve la url de la imagen para poder usarse o te devuelve null y se te queda el icono de perfil default a la hora de mostrarse.

```
private fun obtenerFotoPerfil() {  
    val userId = auth.currentUser?.uid ?: return  
    val storageRef = storage.reference.child( pathString: "profile_images/$userId.jpg")  
  
    storageRef.downloadUrl  
        .addOnSuccessListener { uri ->  
            _profileImageUrl.value = uri.toString()  
        }  
        .addOnFailureListener {  
            _profileImageUrl.value = null  
        }  
}
```

Método que se descarga la imagen de perfil de Storage

Una de las partes más importante ha destacar es el método **guardarRecordatorio()** que es el método que crea el recordatorio. El método empieza verificando que los campos importantes como el uid, título y descripción no son nulos después obtiene los colores que se pueden seleccionar y de esos obtienes el color que ha elegido el usuario junto el color de la categoría que ha elegido si ha elegido una . Después los pasa a string y crea el objeto recordatorio y después mediante una corrutina verifica que se haya creado la subcolección y guarda el recordatorio en esa subcoleccion junto la ruta del recordatorio en el array de **recordatorios_disponibles**. En el caso que no falle se ejecuta el método para programar la notificación del recordatorio 2 horas antes.

```
fun guardarRecordatorio(
    onSuccess: () -> Unit,
    onFailure: (Exception) -> Unit
) {
    val uid = auth.currentUser?.uid
    if (uid == null) {
        onFailure(Exception("Usuario no autenticado"))
        return
    }
    if (titulo.isBlank()) {
        onFailure(Exception("El título no puede estar vacío."))
        return
    }
    if (descripcion.isBlank()) {
        onFailure(Exception("La descripción no puede estar vacía."))
        return
    }

    val colorSeleccionado: Color = colores[colorIndex.toInt()]
    val colorHex = String.format("#%06X", colorSeleccionado.toArgb())

    val prioridadSeleccionada = prioridades[prioridadIndex.toInt()]

    val colorCategoria = categoriaSeleccionada?.let { key ->
        categoriasDisponibles[key]?.let { color ->
            String.format("#%06X", color.toArgb())
        }
    } ?: ""
```

```
val recordatorio = Recordatorio(  
    titulo = titulo,  
    descripcion = descripcion,  
    fecha = fechaHora,  
    prioridad = prioridadSeleccionada,  
    color = colorHex,  
    color_de_la_categoria = colorCategoria,  
    esFavorito = false,  
    esEditable = esEditable,  
    esta_Compartido = false,  
    lista_compartidos = emptyList(),  
    compartidoPor = uid  
)  
  
viewModelScope.launch(Dispatchers.IO) {  
    try {  
        val userDoc = firestore.collection( collectionPath "Users").document(uid)  
        val userSnapshot = userDoc.get().await()  
        if (!userSnapshot.exists()) {  
            userDoc.set(hashMapOf("creado" to true)).await()  
        }  
  
        val docRef = userDoc.collection( collectionPath "Reminders").document()  
        val recordatorioConId = recordatorio.copy(id = docRef.id)
```

```
docRef.set(recordatorioConId).await()

userDoc.update( field: "recordatorios_disponibles", FieldValue.arrayUnion(docRef)).await()
categoriaSeleccionada?.let { categoriaNombre ->
    val categoriaQuery = firestore.collection( collectionPath "Categories")
        .whereEqualTo( field: "nombre", categoriaNombre)
        .get()
        .await()

    if (!categoriaQuery.isEmpty) {
        val categoriaDoc = categoriaQuery.documents.first().reference
        categoriaDoc.update( field: "recordatorios_pertenecientes", FieldValue.arrayUnion(docRef)).await()
    } else {
        Log.w( tag: "Categorias", msg: "No se encontró la categoría con nombre '$categoriaNombre'")
    }
}

firestore.collection( collectionPath "Notification").add(
    mapOf(
        "descripcion" to descripcion,
        "usuario" to userDoc,
        "recordatorio" to docRef,
        "fechaCreacion" to FieldValue.serverTimestamp()
    )
).await()
```

```
        withContext(Dispatchers.Main) {
            programarNotificacion(context, recordatorioConId)
            onSuccess()
        }
    } catch (e: Exception) {
        withContext(Dispatchers.Main) {
            onFailure(e)
        }
    }
}
}
```

Método que crea el recordatorio y lo programa

6. DESPLIEGUE DEL PRODUCTO

Para desplegar la aplicación voy a hacerlo mediante apk que posteriormente se instalará en el dispositivo móvil

7. PRESUPUESTO, RIESGOS, VIABILIDAD

En este caso este proyecto sería viable y este sería el presupuesto necesario:

Actividad realizada de la App	Horas empleadas	Coste
Planificación del proyecto	5	125
Diseño de la interfaz	7	175
Desarrollo de la interfaz	10	250
Desarrollo del inicio de sesión	4	100
Desarrollo de las funcionalidades de los recordatorios y categorías	12	300
Desarrollo de las funciones sociales	11	275
Optimización de la App	3	75
Total necesario	52	1300

8. DIFICULTADES ENCONTRADAS Y RESOLUCIÓN

- La primera dificultad encontrada es el siguiente:

Error de incompatibilidades por el hilt:Execution failed for task ':app:hiltAggregateDepsDebug'. > A failure occurred while executing dagger.hilt.android.plugin.task.AggregateDepsTask\$WorkerAction > 'java.lang.String com.squareup.javapoet.ClassName.canonicalName()'

Esta dificultad impidió construir la aplicación ya que eran incompatibles la versión de kotlin con kapt que era el compilador del inyector de dependencias dagger hilt. Para arreglar esta dificultad cambie a un compilador más actual llamado **kps** para evitar las incompatibilidades dadas.

- Lo siguiente que encontré fue que al intentar almacenar un color en la base de datos no se me guardaba en el formato adecuado. Para poder arreglarlo pase el Int del color seleccionado a Argb y luego a un formato de string igual al formato de hexadecimal con este método :
String.format("#%06X", colorSeleccionado.toArgb())

- Otra fue que no encontraba los recordatorios para mostrarlos en la pantalla. Lo que hice para solucionarlo fue ver mediante **Logs** como se cargaban los recordatorios y si estaban bien

puesto la ruta para llamar los recordatorios creados por el usuario y vi que no la había puesto correctamente la corregí y ya funcionaba. Posteriormente cambie el método para que saque las rutas a los documentos del campo **recordatorios_disponibles** para que me mostrase tanto los recordatorios creados por ti como los compartidos por otros usuarios.

• Una dificultad bastante relevante fue que no podía programar las notificaciones ya que aunque pidiese permisos no me salían para aceptar el permiso. Para solucionar este problema agregue lo siguiente:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        notificationPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS)
    }

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
        val alarmManager = getSystemService(Context.ALARM_SERVICE) as AlarmManager
        if (!alarmManager.canScheduleExactAlarms()) {
            val intent = Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM)
            startActivity(intent)
            Toast.makeText(context: this, text: "No se pueden programar alarmas exactas", Toast.LENGTH_LONG).show()
        }
    }
}
```

Método que revisa si están permitidos los permisos

Este código lo que hace es que si no tienes el permiso de notificaciones te abre la pestaña de permisos de las configuraciones para que des permiso a recibir notificaciones y al aceptarlo y volver a la aplicación te sale otra pestaña para asegurar de que quieres permitir notificaciones. En el caso de que no permitas las notificaciones te saldrá un mensaje de que no se pueden programar alarmas exactas

9. CONCLUSIONES

El desarrollo de aplicaciones móviles es bastante complejo a la hora de desarrollar la aplicación ya que hay muchos factores a tener en cuenta.

La base de datos en la nube es bastante más eficiente frente a las locales porque tienen mejor rendimiento. Además de ser más fácil de revisar la información contenida en ella aunque tiene el inconveniente de que se requiera una conexión a la red para el funcionamiento de la aplicación.

Es muy recomendable utilizar un inyector de dependencias porque simplifica mucho el código y te ahorras pasar objetos y viewmodels entre pantallas aunque a la hora de aprender es bastante confuso al principio.

El uso de herramientas en la nube como Firebase facilita muchos ámbitos de la aplicación como a la hora de que aparezcan craseos o la autenticación de la aplicación.

10. REFERENCIAS BIBLIOGRÁFICAS

Google (s.f) Precios de Firestore <https://cloud.google.com/firestore/pricing?hl=es-419>

Google (s.f) Planes de precios de Firebase <https://firebase.google.com/pricing?hl=es-419>

OpenAI. (s.f) ChatGPT <https://chatgpt.com/>

Programación by AristiDevs(3 oct 2024) FIREBASE desde CERO en ANDROID con KOTLIN y JETPACK COMPOSE <https://www.youtube.com/watch?v=LxABxtwhrDE&t=4789s>

Stack overflow (s.f) Stack Overflow <https://stackoverflow.com/>